LDOCX Technical Specification v2.5.0

Living Document Architecture, Binary Container & AST Standard | LDOCX Standard v2.5.0

OFFICIAL TECHNICAL SPECIFICATION STANDARD - RFC-LDOCX-2026

1. Executive Specification Overview

The Living Document XML/Extended (LDOCX) specification defines an open, platform-agnostic, reactive binary container format for interactive documents. Unlike static paper-emulating formats such as PDF or fixed presentations (PPTX), LDOCX files are self-contained reactive software packages containing structured Abstract Syntax Trees (ASTs), WebGL spatial shaders, tactile paper physics, and live client-side execution sandboxes.

Specification Standard:	`LDOCX Unified Format v2.5.0`
Standard MIME Type:	`application/vnd.ldocx (+json)`
Default Compression:	`ZIP Container / DEFLATE Level 6`
AST Schema Version:	`https://schema.ldocx.org/v2.5/document.json`
Execution Model:	`Zero-Trust Hermetic Client-Side Sandbox`

2. Key Design Principles

> Hermetic Zero-Server Dependency: Documents render 100% client-side with no mandatory backend AP
> Living Spatial AST: Interactive 3D models, particle dynamics, and water ripple physics embedded
> Tactile Physicality: Smooth corner-lift page physics and responsive fluid motion dynamics.
> Graceful Lenient Recovery: Damaged archives are quarantined without corrupting sibling content.

Package Container & ZIP File Architecture

Physical Directory Layout, File Naming, & Integrity Checks | LDOCX Standard v2.5.0

3. Package Container Layout (.ldocx)

An .ldocx package is an unencrypted, standardized ZIP archive with standard compression. The root directory contains the manifest, metadata descriptors, individual page AST representations, and asset directories.

[CODE SPECIFICATION: Package Directory Tree]

```
document.ldocx (ZIP Archive)
|-- manifest.json            <- Global metadata, page sequence, schema declaration
|-- spec.json                <- Unified serialized AST (for single-pass parsers)
|-- pages/                   <- Individual page definitions
|   |-- page_001.json        <- Page 1 AST nodes, blocks & floating text arrays
|   |-- page_002.json        <- Page 2 AST nodes, blocks & floating text arrays
|   '-- page_NNN.json        <- Page N AST representation
|-- assets/                  <- Embedded binary assets
|   |-- models/              <- 3D GLTF / GLB / OBJ binary meshes
|   |-- images/              <- High-fidelity webp / png / svg image assets
|   '-- audio/               <- Ambient atmospheric soundtrack loops
|-- scripts/                 <- Sandboxed React / JSX interactive components
'-- checksum.sha256          <- Cryptographic validation signature
```

4. manifest.json Specification

The manifest.json file must be located in the archive root and must strictly conform to the following schema definition:

[CODE SPECIFICATION: manifest.json Schema Definition]

```
{
  "ldoc_version": "2.5.0",
  "id": "doc_9f83a02c",
  "title": "Living Document Technical Report",
  "author": "Chief Systems Architect",
  "lang": "en-US",
  "created_at": "2026-09-04T12:00:00.000Z",
  "theme": "velocity",
  "default_presentation_mode": "paper",
  "page_count": 2
}
```

Abstract Syntax Tree (AST) Specification

Universal Block Model, Type Discriminators & Attributes | LDOCX Standard v2.5.0

5. Universal Block Node Model

Every presentation element inside an LDOCX page is modeled as a declarative AST node. The parser validates the block discriminator and delegates rendering to the active engine layer.

Block Type	Primary Attributes	Interactive Engine Action
`heading`	level (1-6), text	In-place WYSIWYG editing, TOC indexer
`paragraph`	text, formatting	Inline rich-text editor with spellcheck
`table`	headers[], rows[][]	Sortable, filterable reactive grid
`code`	language, code, run	Syntax highlighted block with code runner
`3d_model`	mesh_template, mode	Three.js WebGL orbit controls & shaders
`water_effect`	intensity, speed	Navier-Stokes fluid ripple dynamics
`particles`	preset (stardust/embers)	GPU particle emitter with mouse repulsion
`jsx_canvas`	code, initial_state	Isolated sandbox runtime with React state
`form`	fields[], submit_url	Live user data capture with validation
`button`	label, action, price	Stripe checkout modal & state triggers

6. Page AST Schema (pages/page_XXX.json)

```
[ CODE SPECIFICATION: Page AST Data Contract ]
 {
   "id": "page_001",
   "page_number": 1,
   "title": "Introduction to Living Documents",
   "blocks": [
     { "id": "blk_1", "type": "heading", "level": 1, "text": "Living Architecture" },
     { "id": "blk_2", "type": "water_effect", "title": "Fluid Waves" }
   ],
   "floating_texts": [
     { "id": "ft_1", "x": 120, "y": 80, "text": "Callout Note", "fontSize": 14 }
   ]
 }
```

In-Canvas Free Text & Floating Typography

Arbitrary 2D Positioning, Typography Control & Serialization | LDOCX Standard v2.5.0

7. Free-Text Layout Engine

LDOCX introduces DOCX-grade absolute floating typography directly onto the document canvas. Unlike standard HTML flow layouts where text is constrained to block stacking, floating text boxes can be positioned at any arbitrary (X, Y) coordinate on the slide surface.

Property	Type & Values	Behavioral Role
`id`	String (UUID/Hash)	Unique identifier for change tracking
`x, y`	Float (px)	Absolute coordinate relative to slide origin (top-le
`fontFamily`	Sans, Cinzel, Serif, Mono	Applied web font family styling
`fontSize`	12px - 48px	Direct typography scale adjustment
`color`	Hex / RGBA (#f59e0b)	Primary text color foreground
`bgStyle`	none, glass, aura, dark	Container background backdrop blur / border
`formatting`	bold, italic, underline	Rich text inline formatting decorators
`dimensions`	width, height (px)	Auto-wrapping box dimensions with resizer

8. Serialization & Backward Compatibility

Floating text nodes are serialized under the "floating_texts" array of each page in the .ldocx package. Legacy parsers that do not support floating texts safely ignore this array while preserving all standard body blocks.

Security Hardening, Sandbox & Resilient Recovery

Hermetic Execution, CSP Rules, Zip Slip Defense & Quarantining | LDOCX Standard v2.5.0

9. Zero-Trust Hermetic Sandbox Architecture

To ensure untrusted .ldocx packages cannot execute arbitrary code or compromise the host machine, the LDOCX viewer enforces strict hermetic sandboxing:

- Iframe Sandbox Directives: Interactive widgets run inside iframe sandboxes with allow-scripts b
- Zip-Slip Traversal Prevention: The ZIP extraction engine rejects any archive entry containing d
- Content Security Policy (CSP): Scripts are strictly isolated from network fetching of external
- XSS Sanitization: All HTML user inputs and imported texts are processed through DOMPurify and s

10. Lenient Parsing & Corrupted Block Quarantine

When an archive experiences bit rot or partial corruption, the LDOC Parser operates in Lenient Recovery Mode. Corrupted blocks are quarantined into safe placeholder nodes (blk_quarantine_XXX) while valid sibling blocks and pages render uninterrupted.